

第2章 线性表

主讲教师：黄绍辉



线性结构是最常用、最简单的一种数据结构。而线性表是一种典型的线性结构。其基本特点是线性表中的数据元素是有序且是有限的。在这种结构中：

- ① 存在一个唯一的被称为“第一个”的数据元素；
- ② 存在一个唯一的被称为“最后一个”的数据元素；
- ③ 除第一个元素外，每个元素均有唯一的一个直接前驱；
- ④ 除最后一个元素外，每个元素均有唯一的一个直接后继。



2.1 线性表的类型定义

线性表(**Linear List**)：是由 $n(n \geq 0)$ 个数据元素(结点) $a_1, a_2, \dots, a_n$ 组成的有限序列。该序列中的所有结点具有相同的数据类型。其中数据元素的个数 n 称为线性表的长度。

当 $n=0$ 时，称为空表。

当 $n>0$ 时，将非空的线性表记作： $(a_1, a_2, \dots, a_n)$

a_1 称为线性表的第一个(首)结点， a_n 称为线性表的最后一个(尾)结点。 $a_1, a_2, \dots, a_{i-1}$ 都是 $a_i(2 \leq i \leq n)$ 的**前驱**，其中 a_{i-1} 是 a_i 的**直接前驱**； $a_{i+1}, a_{i+2}, \dots, a_n$ 都是 $a_i(1 \leq i \leq n-1)$ 的**后继**，其中 a_{i+1} 是 a_i 的**直接后继**。



线性表的逻辑结构

- ◆ 线性表中的数据元素 a_i 所代表的具体含义随具体应用的不同而不同，在线性表的定义中，只不过是一个抽象的表示符号。
- ◆ 线性表中的结点可以是单值元素(每个元素只有一个数据项)。

例1: 26个英文字母组成的字母表: (A, B, C, ..., Z)

例2: 某校从1978年到1983年各种型号的计算机拥有量的变化情况: (6, 17, 28, 50, 92, 188)

例3: 一副扑克的点数 (2, 3, 4, ..., J, Q, K, A)

◆ 线性表中的**结点**可以是**记录型**元素，每个元素含有多个数据项，每个项称为结点的一个域。每个元素有一个可以唯一标识每个结点的**数据项组**，称为**关键字**。

例4：某校2001级同学的基本情况：{(‘2001414101’，‘张三’，‘男’，06/24/1983)，…，(‘2001414102’，‘李四’，‘女’，08/12/1984)}

◆ 若线性表中的结点是**按值**(或按关键字值)由小到大(或由大到小)**排列**的，称线性表是有序的。

◆ 线性表长度可根据需要增长或缩短。

◆ 对线性表的数据元素可以访问、插入和删除。

线性表的抽象数据类型定义

ADT List{

数据对象: $D = \{ a_i \mid a_i \in \text{ElemSet}, i=1,2,\dots,n, n \geq 0 \}$

数据关系: $R = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2,3,\dots,n \}$

基本操作:

InitList(&L) 这个符号**&**表示C++的**引用**，下页有个示例。

操作结果: 构造一个空的线性表L;

...



C++的引用，相当于形参实参内存共享，很适合用来改变实参的值。

```
#include <stdio.h>
```

```
void f1(int i, int *q)
```

```
{ //修改形参，实参不影响
```

```
    i++; q++;
```

```
}
```

```
void f2(int &i, int *&q)
```

```
{ //修改形参，实参同时改变
```

```
    i++; q++;
```

```
}
```

引用比return更方便传出结果

```
int main()
```

```
{
```

```
    int a=10,b[]={20,30};
```

```
    int *p=b;
```

```
    f1(a,p);
```

```
    printf("a=%d, *p=%d\n",a,*p);
```

```
    f2(a,p);
```

```
    printf("a=%d, *p=%d\n",a,*p);
```

```
    return 0;
```

```
}
```

a=10, *p=20

a=11, *p=30

ListLength(L)

初始条件：线性表L已存在；

操作结果：若L为空表，则返回**TRUE**，否则返回**FALSE**；

...

GetElem(L, i, &e)

初始条件：线性表L已存在， $1 \leq i \leq \text{ListLength}(L)$ ；

操作结果：用e返回L中第i个数据元素的值；

ListInsert (&L, i, e)

初始条件：线性表L已存在， $1 \leq i \leq \text{ListLength}(L)$ ；

操作结果：在线性表L中的第i个位置插入元素e；

...

} ADT List

例2-1 假设利用两个线性表LA和LB分别表示两个集合A和B，求一个新集合 $A=A \cup B$ 。

```
void union(List &La, List Lb) { //La变 , Lb不变  
    //将所有在Lb中但不在La中的元素插入La  
    La_len=ListLength(La); Lb_len=ListLength(Lb);  
    for(i=1;i<=Lb_len;i++) {  
        GetElem(Lb, i, e);  
        if (!LocateElem(La, e, equal)  
            ListInsert(La, ++La_len, e)  
    }  
} //union
```

例2-2 线性表LA和LB的元素按值非递减有序排列，现要求将La和Lb归并为一个新的线性表Lc，且Lc按值非递减有序排列。

```
void MergeList(List La, List Lb, List &Lc) {  
    InitList(Lc); i=j=1; k=0;  
    La_len=ListLength(La); Lb_len=ListLength(Lb);  
    while((i<=La_len) && (j<=Lb_len)) { //La和Lb均非空  
        GetElem(La, i, ai); GetElem(Lb, j, bj);  
        if (ai<=bj) { ListInsert(Lc, ++k, ai);++i; }  
        else { ListInsert(Lc, ++k, bj);++j; }  
    }  
    while(i<=La_len){GetElem(La, i++, ai); ListInsert(Lc, ++k, ai);}  
    while(j<=Lb_len){GetElem(Lb, j++, bj); ListInsert(Lc, ++k, bj);}  
} //MergeList
```

2.2 线性表的顺序表示和实现

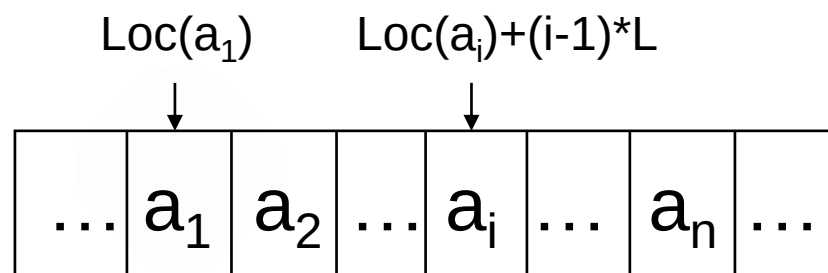
顺序存储：把线性表的结点**按逻辑顺序依次**存放在一组地址连续的**存储单元**里。用这种方法存储的线性表简称顺序表。

顺序存储的线性表的特点：

- ◆ 线性表的逻辑顺序与物理顺序一致；
- ◆ 数据元素之间的关系是以元素在计算机内“**物理位置相邻**”来体现。

设有非空的线性表： $(a_1, a_2, \dots, a_n)$ 。顺序存储如图所示。





线性表的顺序存储表示

在具体的机器环境下：设线性表的每个元素需占用 L 个存储单元，以所占的第一个单元的存储地址作为数据元素的存储位置。则线性表中第 $i+1$ 个数据元素的存储位置 $LOC(a_{i+1})$ 和第 i 个数据元素的存储位置 $LOC(a_i)$ 之间满足下列关系：

$$LOC(a_{i+1}) = LOC(a_i) + L$$

线性表的第 i 个数据元素 a_i 的存储位置为：

$$LOC(a_i) = LOC(a_1) + (i-1) * L$$

在高级语言(如C语言)环境下：数组具有随机存取的特性，因此，可以借助数组来描述顺序表。除了用数组来存储线性表的元素之外，顺序表还应该具有表示线性表的长度属性。

下面用结构类型来定义顺序表类型。

```
#define LIST_INIT_SIZE 100 //空间初始分配量
```

```
#define LISTINCREMENT 10 //空间分配增量
```

```
typedef struct
```

```
{ ElemType *elem; //存储空间基址
```

```
    int length; //当前长度
```

```
    int listsize; //当前分配容量
```

```
} SqList ;
```


- ◆ 顺序存储结构中，很容易实现线性表的一些操作：初始化、赋值、查找、修改、插入、删除、求长度等。

以下将对几种主要的操作进行讨论。

1 顺序线性表初始化

```
Status InitList_Sq ( SqList &L )
```

```
{ L.elem=( ElemType *)malloc( LIST_INIT_SIZE *  
  sizeof( ElemType ) );
```

```
  if ( !L.elem ) exit(OVERFLOW);
```

```
  L.length=0;   L.listsize=LIST_INIT_SIZE;
```

```
  return OK;
```

```
}
```



2 顺序线性表的插入

在线性表 $L = (a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$ 中的第 i ($1 \leq i \leq n$) 个位置上插入一个新结点 e , 使其成为线性表:

$$L = (a_1, \dots, a_{i-1}, e, a_i, a_{i+1}, \dots, a_n)$$

实现步骤

- (1) 将线性表 L 中的第 i 个至第 n 个结点后移一个位置。
- (2) 将结点 e 插入到结点 a_{i-1} 之后。
- (3) 线性表长度加1。



```
Status ListInsert_Sq(Sqlist &L , int i , ElemType e) {  
    if ( i<1 || i>L.length+1) return ERROR ;  
    if (L.length>=L.listsize) //空间已满  
        { //重新分配内存 , 代码参考课本 }  
    q = &(L.elem[i-1]);  
    for(p=&(L.elem[L.length-1]); p>=q; --p) *(p+1)=*p;  
        /* 插入位置以后的所有结点右移 */  
    *q=e; /* 在i-1位置插入结点 */  
    ++L.length ;  
    return OK ;  
}
```

时间复杂度分析

在线性表L中的第i个元素之前插入新结点，其时间主要耗费在表中结点的移动操作上，因此，可用结点的移动来估计算法的时间复杂度。

设在线性表L中的第i个元素之前插入结点的概率为 p_i ，不失一般性，设各个位置插入是等概率，则 $p_i=1/(n+1)$ ，而插入时移动结点的次数为 $n-i+1$ 。

总的平均移动次数： $E_{\text{insert}} = \sum p_i * (n-i+1) \quad (1 \leq i \leq n)$

$\therefore E_{\text{insert}} = 1/(n+1) * (n + n-1 + n-2 + \dots + 1) = n/2$ 。

即在顺序表上做插入运算，平均要移动表上一半结点。当表长n较大时，算法的效率相当低。因此算法的平均时间复杂度为 $O(n)$ 。

顺序线性表的删除

在线性表 $L=(a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$ 中删除结点 $a_i (1 \leq i \leq n)$ ，使其成为线性表：

$$L = (a_1, \dots, a_{i-1}, a_{i+1}, \dots, a_n)$$

实现步骤

- (1) 将线性表L中的第 $i+1$ 个至第 n 个结点依次向前移动一个位置。
- (2) 线性表长度减1。



```
Status ListDelete_Sq(Sqlist &L , int i, ElemType &e) {  
    if (i<1 || i>L.length) return ERROR;  
    p=&(L.elem[i-1]);  
    e=*p;  
    q=L.elem+L.length-1;  
    for(++p;p<=q;++p) *(p-1)=*p;  
    --L.length;  
    return OK;  
}
```


时间复杂度分析

删除线性表L中的第i个元素，其时间主要耗费在表中结点的移动操作上，因此，可用结点的移动来估计算法的时间复杂度。

设在线性表L中删除第i个元素的概率为 p_i ，不失一般性，设删除各个位置是等概率，则 $p_i=1/n$ ，删除时移动结点的次数为 $n-i$ 。

则总的平均移动次数： $E_{\text{delete}} = \sum p_i * (n-i) \quad (1 \leq i \leq n)$

$\therefore E_{\text{delete}} = 1/n * (n-1 + n-2 + \dots + 1) = (n-1)/2$ 。

即在顺序表上做删除运算，平均要移动表的一半结点。当表长 n 较大时，算法的效率相当低。因此算法的平均时间复杂度为 $O(n)$ 。

顺序线性表的查找定位

查找元素最简单的办法，就是让e和L中的数据元素逐个比较。
compare是函数指针，需传入用于比较两个元素的函数名。

```
int LocateElem_Sq(Sqlist L, ElemType e,  
                  Status(*compare)(ElemType, ElemType)) {  
    i=1;  
    p=L.elem;  
    while (i<=L.length && !(*compare)(*p++, e)) ++i;  
    if (i<=L.length) return i;  
    else return 0;  
}
```



将顺序表La和Lb合并为一个新的顺序表Lc。

```
void MergeList_Sq(SqList La, SqList Lb, SqList &Lc) {  
    pa=La.elem; pb=Lb.elem;  
    Lc.listsize=Lc.length=La.length+Lb.length;  
    pc=Lc.elem=(ElemType*)malloc(Lc.listsize *  
sizeof(ElemType));  
    if (!Lc.elem) exit(OVERFLOW);  
    pa_last=La.elem+La.length-1;  
    pb_last=Lb.elem+Lb.length-1;
```

接上页

```
while((pa<=pa_last) && (pb<=pb_last)) { //La和Lb非空
    if (*pa<=*pb) *pc++ = *pa++;
    else *pc++ = *pb++;
}
while(pa<=pa_last) *pc++ = *pa++;
while(pb<=pb_last) *pc++ = *pb++;
} //MergeList_Sq
```

- 上述算法的时间复杂度为： $O(La.length + Lb.length)$
- 以线性表表示集合并进行集合的各种运算，应先对表中元素进行排序。

2.3 线性表的链式表示和实现

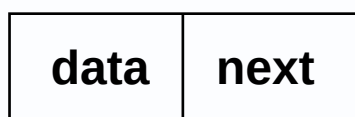
链式存储：用一组任意的存储单元存储线性表中的数据元素。用这种方法存储的线性表简称**线性链表**。

存储链表中结点的一组任意的存储单元可以是连续的，也可以是不连续的，甚至是零散分布在内存中的任意位置上的。

链表中结点的逻辑顺序和物理顺序不一定相同。



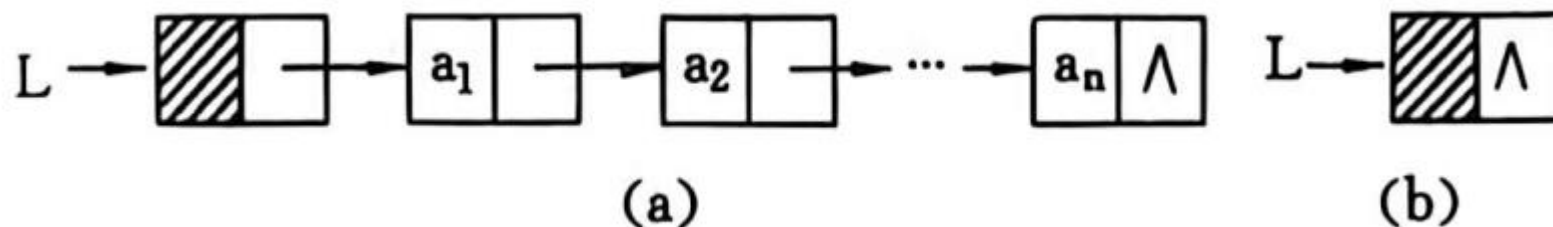
为了正确表示结点间的逻辑关系，在存储每个结点值的同时，还必须存储指示其直接后继结点的地址（或位置），称为指针(pointer)或链(link)，这两部分组成了链表中的结点结构，如图。



data：数据域，存放结点的值。next：指针域，存放结点的直接后继的地址。

链表是通过每个结点的指针域将线性表的n个结点按其逻辑次序链接在一起的。每一个结点只包含一个指针域的链表，称为单链表。

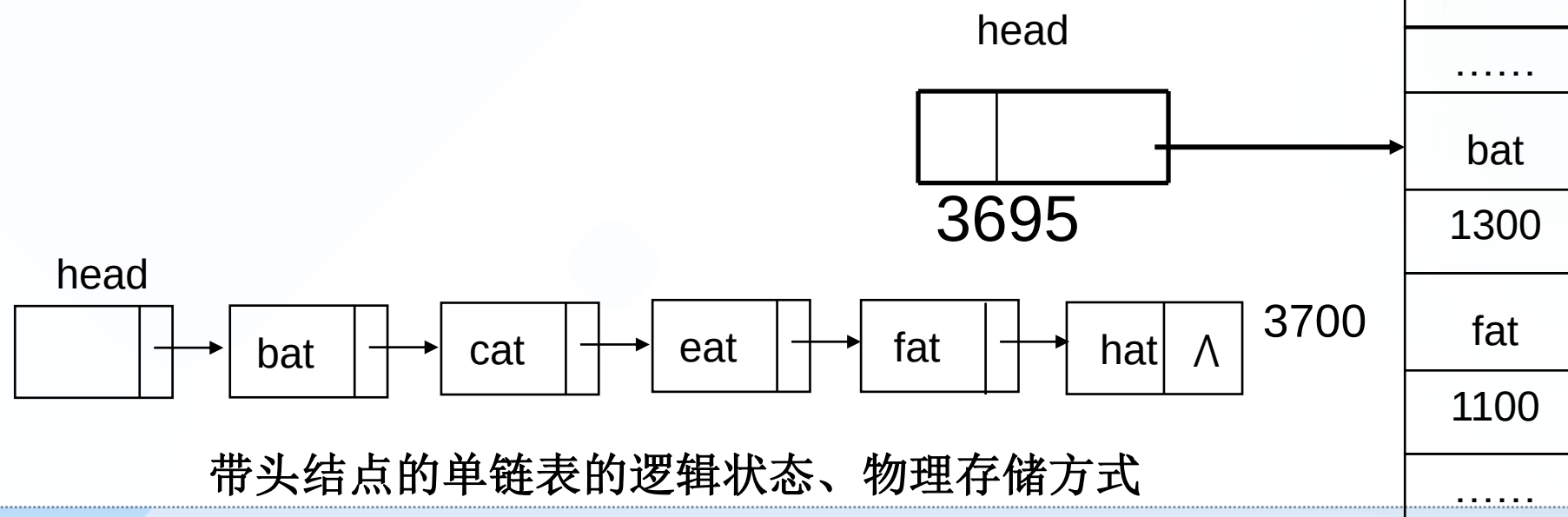
为操作方便，总是在链表的第一个结点之前附设一个头结点（头指针），head指向第一个结点。头结点的数据域可以不存储任何信息（单纯占个位置），也可以存储链表长度等信息。



单链表是由表头唯一确定，因此单链表可以用头指针的名字来命名。

例1、线性表L=(bat, cat, eat, fat, hat)

其带头结点的单链表的逻辑状态和物理存储方式如图所示。



带头结点的单链表的逻辑状态、物理存储方式

1 结点的描述与实现

C语言中用带指针的结构体类型来描述

```
typedef struct LNode
```

```
{ ElemType data; /*数据域，保存结点的值 */
```

```
    struct LNode *next; /*指针域*/
```

```
}LNode, *LinkList; /*结点和结点指针类型 */
```

2 结点的实现

结点是通过动态分配和释放来的实现，即需要时分配，不需要时释放。实现时分别使用C语言提供的标准函数：

`malloc()`，`realloc()`，`sizeof()`，`free()`。



动态分配 `p=(LNode*)malloc(sizeof(LNode));`

函数malloc分配了一个类型为LNode的结点变量的空间，并将其首地址放入指针变量p中。

动态释放 `free(p);`

系统回收由指针变量p所指向的内存区。p必须是最近一次调用malloc函数时的返回值。

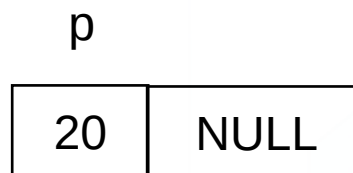
3 最常用的基本操作及其示意图

(1) 结点的赋值

LNode *p;

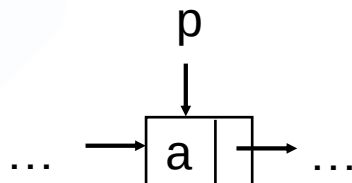
p=(LNode*)malloc(sizeof(LNode));

p->data=20; p->next=NULL ;

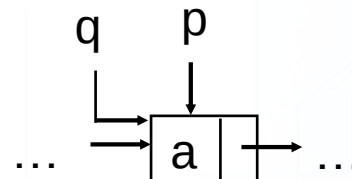


(2) 常见的指针操作

① $q=p;$

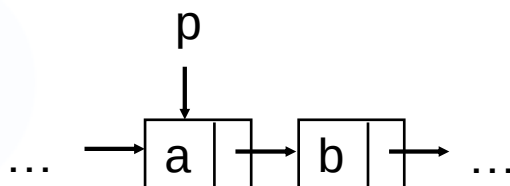


操作前

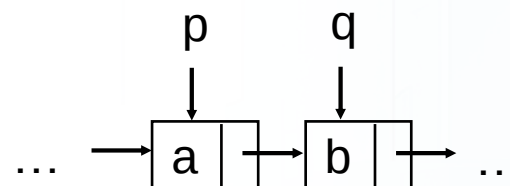


操作后

② $q=p->next;$

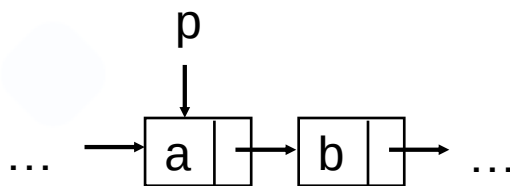


操作前

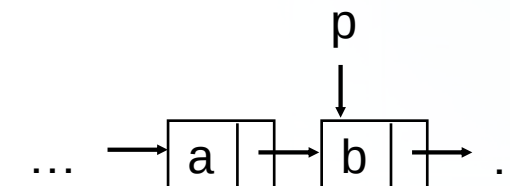


操作后

③ $p=p->next;$



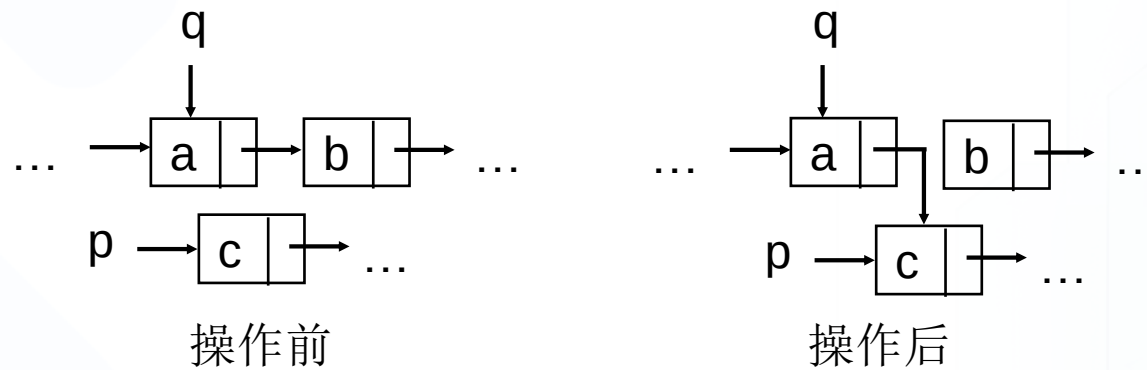
操作前



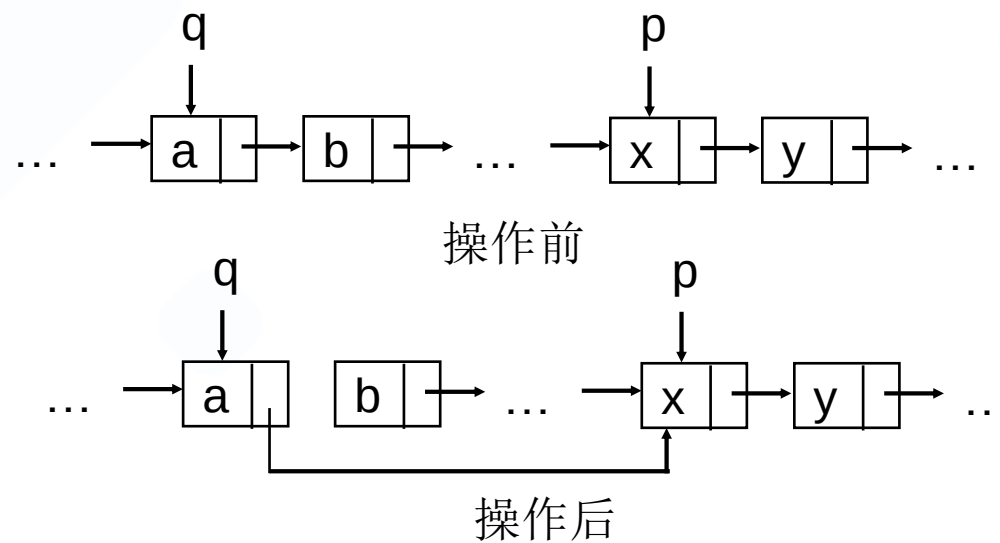
操作后

④ $q \rightarrow \text{next} = p$;

(a)

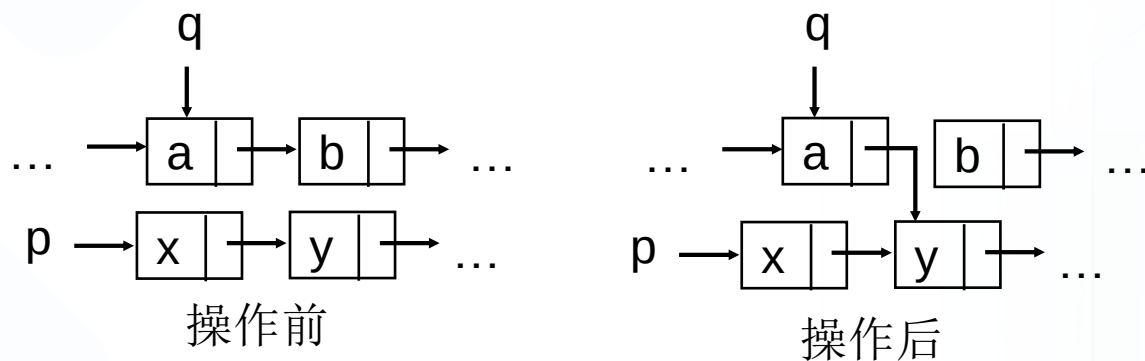


(b)

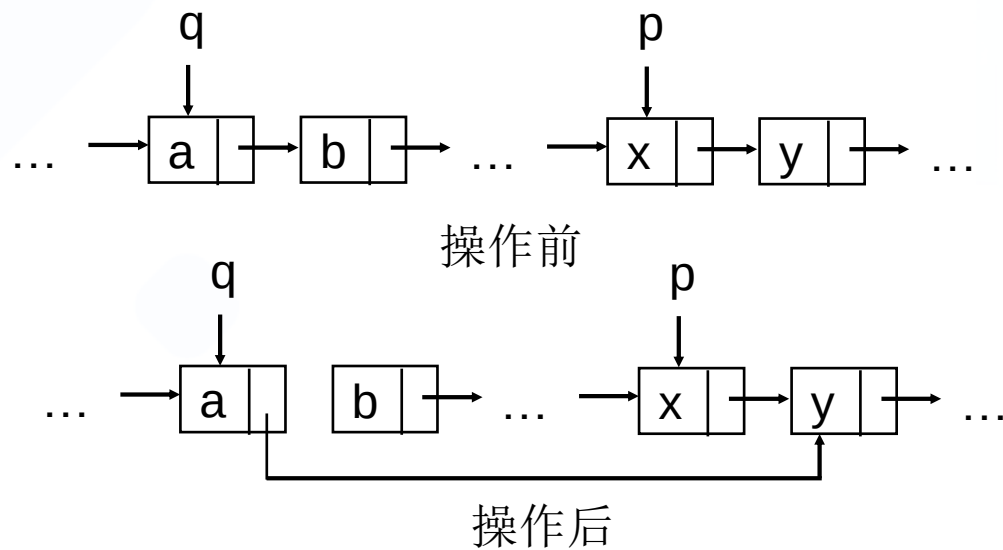


⑤ $q \rightarrow \text{next} = p \rightarrow \text{next}$;

(a)



(b)



单线性链式的基本操作

1 建立单链表

假设线性表中结点的数据类型是整型，现在要创建有 n 个结点的链表。动态地建立单链表的常用方法有如下两种：头插法，尾插法。

(1) 头插法建表

从一个空表开始，重复读入数据，生成新结点，将读入数据存放到新结点的数据域中，然后将新结点插入到当前链表的表头上。即**每次插入的结点都作为链表的第一个结点**。



算法2.11 , 头插法创建**逆序**链表

```
void CreateList_L(LinkList &L, int n) {  
    L=(LinkList)malloc(sizeof(LNode));  
    L->next=NULL;  
    for(i=n;i>0;i--) {  
        p=(LinkList)malloc(sizeof(LNode));  
        scanf(&p->data);  
        p->next=L->next; //注意不是L , L是头结点 , 不算链表  
        L->next=p;  
    }  
} //CreateList_L
```

(2) 尾插入法建表

头插入法建立链表虽然算法简单，但生成的链表中结点的次序和输入的顺序相反。若希望二者次序一致，可采用尾插法建表。该方法是将新结点插入到当前链表的表尾，使其成为当前链表的尾结点。

无论是哪种插入方法，如果要插入建立的单线性链表的结点是 n 个，算法的时间复杂度均为 $O(n)$ 。

对于单链表，无论是哪种操作，只要涉及到钩链（或重新钩链），如果没有明确给出直接后继，钩链（或重新钩链）的次序必须是“先右后左”。

补充算法，尾插法创建顺序链表

```
void CreateList2_L(LinkList &L, int n) {  
    L=q=(LinkList)malloc(sizeof(LNode));  
    L->next=NULL;  
    for(i=n;i>0;i--) {  
        p=(LinkList)malloc(sizeof(LNode));  
        scanf(&p->data);  
        p->next=NULL; //p是尾结点，后继始终是NULL  
        q->next=p;  
        q=p; //q始终在尾结点处  
    }  
} //CreateList2_L
```

2 单链表的查找

(1) **按序号查找** 取单链表中的第*i*个元素。

对于单链表，不能象顺序表中那样直接按序号*i*访问结点，而只能从链表的头结点出发，沿链域next逐个结点往下搜索，直到搜索到第*i*个结点为止。因此，**链表不是随机存取结构**。

设单链表的长度为*n*，要查找表中第*i*个结点，仅当 $1 \leq i \leq n$ 时，*i*的值是合法的。



算法2.8

```
Status GetElem_L(LinkList L, int i, ElemType &e) {  
    p=L->next; j=1;  
    while (p && j<i)  
        { p=p->next; ++j; }  
    if (!p || j>i) return ERROR;  
    e=p->data;  
    return OK;  
} //GetElem_L
```

移动指针p的频率： $i < 1$ 时：0次； $i \in [1, n]$ ： $i-1$ 次； $i > n$ ： n 次。

$\therefore$ 时间复杂度： $O(n)$ 。

(2) 按值查找

按值查找是在链表中，查找是否有结点值等于给定值key的结点。若有，则返回首次找到的值为key的结点的存储位置；否则返回NULL。

查找时从开始结点出发，沿链表逐个将结点的值和给定值key作比较。



算法描述

Status LocateNode_L(LinkList L, ElemType key, LinkList &node) {

p=L->next;

while (p && p->data!=key) p=p->next;

if (p && p->data==key) { node=p; return OK; }

else return ERROR;

}

算法的执行与形参key有关，平均时间复杂度为 $O(n)$ 。

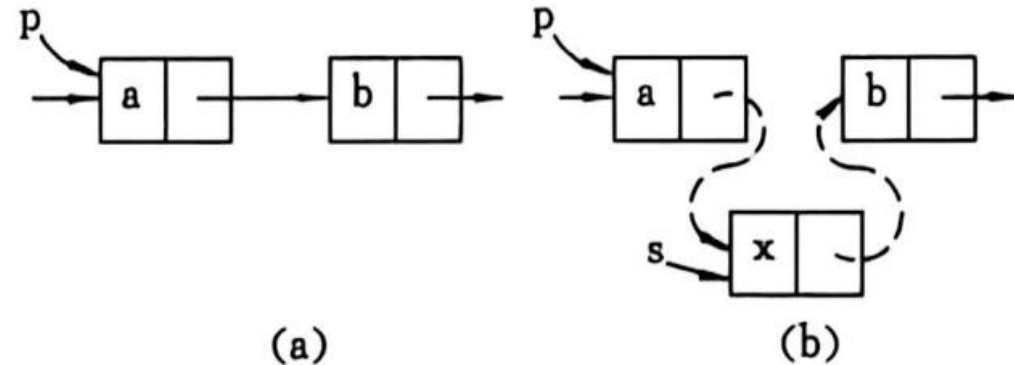
3 单链表的插入

插入运算是将值为 e 的新结点插入到表的第 i 个结点的位置上，即插入到 a_{i-1} 与 a_i 之间。因此，必须首先找到 a_{i-1} 所在的结点 p ，然后生成一个数据域为 e 的新结点 q ， q 结点作为 p 的直接后继结点。



算法2.9

```
Status ListInsert_L(LinkList &L, int i, ElemType e) {  
    p=L; j=0;  
    while ( p && j<i-1) { p=p->next; ++j; }  
    if (!p || j>i-1) return ERROR;  
    s=(LinkList)malloc(sizeof(LNode));  
    s->data=e; s->next=p->next;  
    p->next=s;  
    return OK;  
}
```



设链表的长度为 n ，合法的插入位置是 $1 \leq i \leq n$ 。算法的时间主要耗费在移动指针 p 上，故时间复杂度亦为 $O(n)$ 。

4 单链表的删除

(1) 按序号删除

删除单链表中的第 i 个结点。

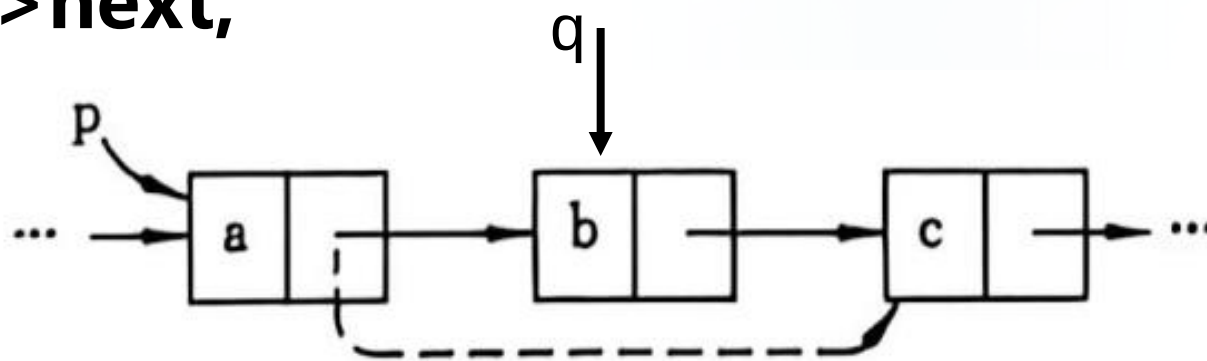
为了删除第 i 个结点 a_i ，必须找到结点的存储地址。该存储地址是在其直接前趋结点 a_{i-1} 的next域中，因此，必须首先找到 a_{i-1} 的存储位置 p ，然后令 $p \rightarrow \text{next}$ 指向 a_i 的直接后继结点，即把 a_i 从链上摘下。最后释放结点 a_i 的空间，将其归还给“**存储池**”。

设单链表长度为 n ，则删去第 i 个结点仅当 $1 \leq i \leq n$ 时是合法的。则当 $i = n + 1$ 时，虽然被删结点不存在，但其前趋结点却存在，是终端结点。故判断条件之一是 $p \rightarrow \text{next} \neq \text{NULL}$ 。



算法2.10

```
Status ListDelete_L(LinkList &L, int i, ElemType &e) {  
    p=L; j=0;  
    while ( p->next && j<i-1)  
        { p=p->next; ++j; }  
    if (!(p->next) || j>i-1) return ERROR;  
    q=p->next; p->next=q->next;  
    e=q->data; free(q);  
    return OK;  
} //ListDelete_L
```



此算法的时间复杂度也是 $O(n)$ 。

(2) 按值删除

删除单链表中值为key的第一个结点。

与按值查找相类似，首先要查找值为key的结点是否存在？若存在，则删除；否则返回NULL。



算法描述，删除data为key的第一个结点

```
Status ListDelete2_L(LinkList &L, ElemType key) {  
    p=L;  
    while ( p->next && p->next->data != key)  
        { p=p->next; }  
    if (!(p->next) || (p->next->data != key)) return ERROR;  
    q=p->next; p->next=q->next;  
    free(q);  
    return OK;  
} //ListDelete2_L
```

算法的执行与形参key有关，平均时间复杂度为 $O(n)$ 。

从上面的讨论可以看出，链表上实现插入和删除运算，无需移动结点，仅需修改指针。解决了顺序表的插入或删除操作需要移动大量元素的问题。

扩展：

删除单链表中值为key的所有结点。

与按值查找相类似，但比前面的算法更简单。

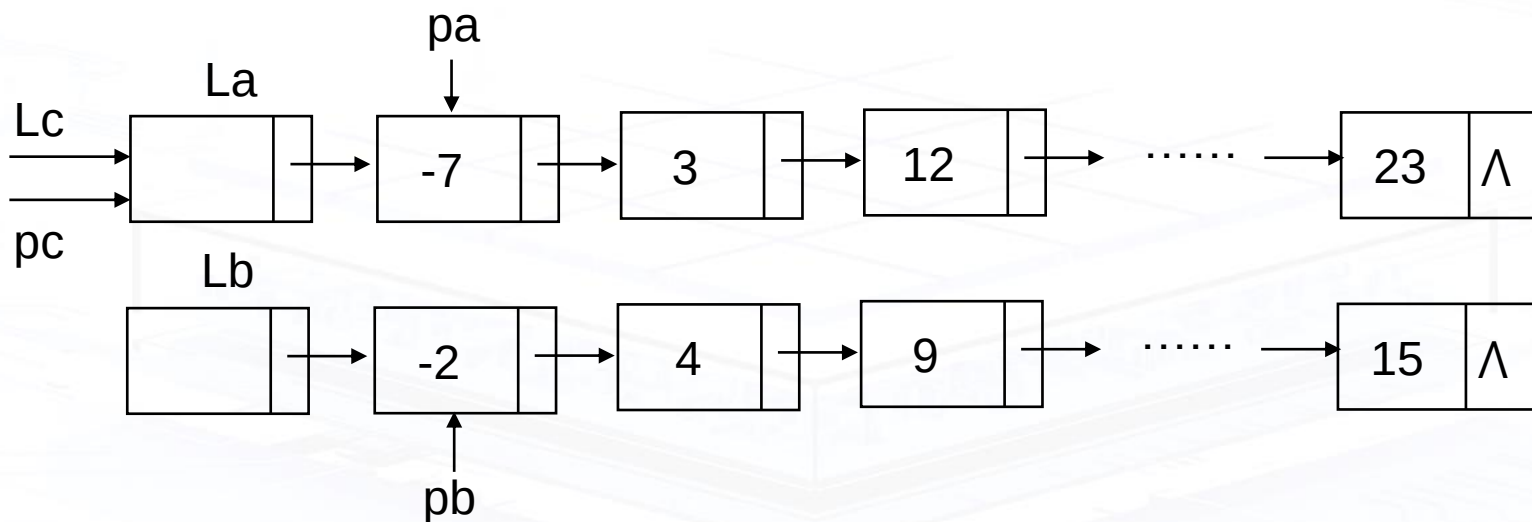
基本思想：从单链表的第一个结点开始，对每个结点进行检查，若结点的值为key，则删除之，然后检查下一个结点，直到所有的结点都检查。

算法描述，删除所有data为key的结点

```
void ListDelete3_L(LinkList &L, ElemType key) {  
    LinkList p=L, q=L->next;  
    while (q) {  
        if (q->data==key)  
            { p->next=q->next; free(q); q=p->next; }  
        else  
            { p=q; q=q->next; }  
    }  
} //ListDelete3_L
```

5 单链表的合并

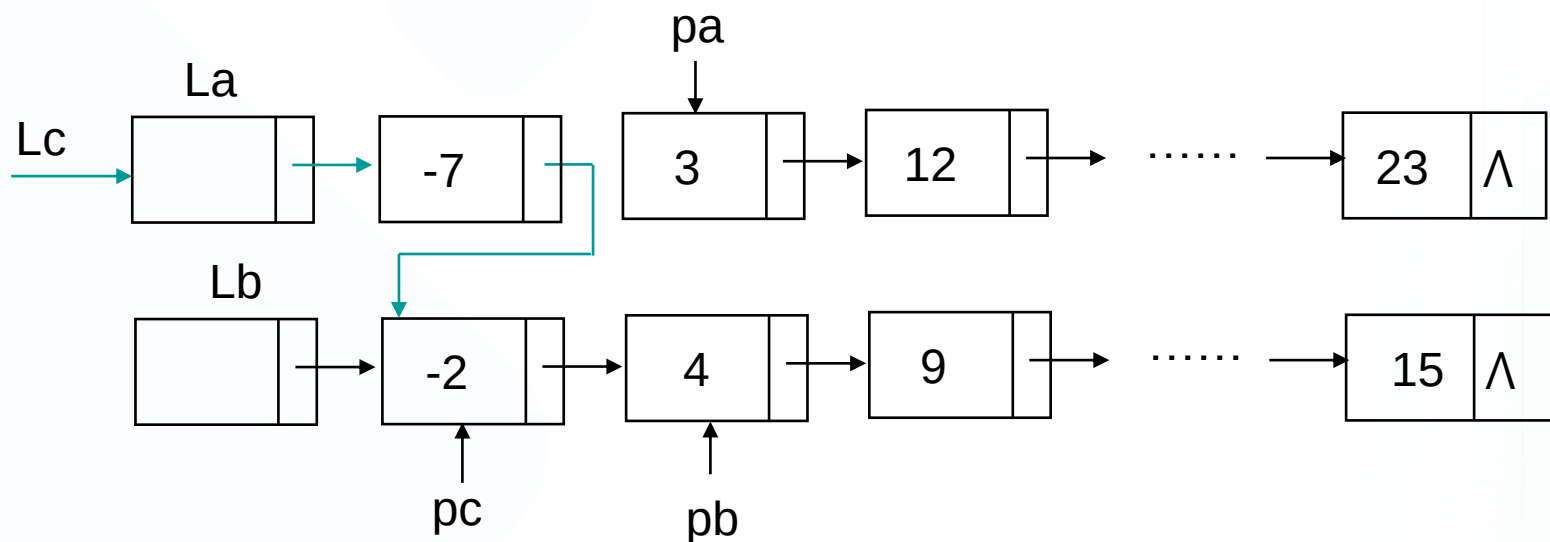
设有两个有序的单链表，它们的头指针分别是La、Lb，将它们合并为以Lc为头指针的有序链表。合并前的示意图如图所示。



两个有序的单链表La，Lb的初始状态



合并了值为-7，-2的结点后示意图如图所示。



合并了值为-7，-2的结点后的状态

算法说明

算法中pa，pb分别是待考察的两个链表的当前结点，pc是合并过程中合并的链表的最后一个结点。

算法2.12

```
void MergeList_L(LinkList &La, LinkList &Lb, LinkList &Lc) {  
    pa=La->next; pb=Lb->next; Lc=pc=La;  
    while (pa && pb) {  
        if (pa->data <= pb->data)  
            { pc->next=pa ; pc=pa ; pa=pa->next ; }  
        else  
            { pc->next=pb ; pc=pb ; pb=pb->next ; }  
    }  
    pc->next=pa?pa:pb; free(Lb);  
} //MergeList_L
```

若La，Lb两个链表的长度分别是m，n，则链表合并的时间复杂度为 $O(m+n)$ 。

6 静态链表

有些编程语言没有类似指针的类型，此时可以用一维数组来描述线性链表，这种链表叫静态链表。

```
#define MAXSIZE 1000
```

```
typedef struct {
```

```
    ElemType data;
```

```
    int cur; //指示下一元素的索引，相当于动态链表的next
```

```
} component, SLinkList[MAXSIZE];
```

数组的一个分量相当于一个结点，cur为0的结点表示尾结点。



静态链表示例

对左侧链表依次执行操作**插入SHI**和**删除ZHENG**两个操作。0号结点为头结点。

0		1
1	ZHAO	2
2	QIAN	3
3	SUN	4
4	LI	5
5	ZHOU	6
6	WU	7
7	ZHENG	8
8	WANG	0
9		
10		

修改前的状态

0		1
1	ZHAO	2
2	QIAN	3
3	SUN	4
4	LI	9
5	ZHOU	6
6	WU	8
7	ZHENG	8
8	WANG	0
9	SHI	5
10		

修改后的状态

- 静态链表的插入和删除无需移动元素，因此效率比使用数组高。
- 左侧例子有个缺陷：**难以确定可以用于存储新结点的位置**。例如索引为**7**和**10**的结点，逻辑上不属于链表，可用于存储新数据，但程序难以判断。
- 为改进这个问题，索引为**0**的结点改为**备用（空闲）链表**的头结点，而实际存储数据的链表头结点，另设一个变量**S**来存储。本例中，可以让**0**号结点的**cur=7**，**S=1**。

例2-3 假设由终端输入集合元素，先输入集合A，再输入集合B，求集合运算 $(A-B) \cup (B-A)$ 的结果。

用静态链表求解，先准备几个辅助函数：

算法2.14，静态链表初始化

```
void InitSpace_SL(SLinkList &space) {  
    for(i=0;i<MAXSIZE-1;++i) space[i].cur=i+1;  
    space[MAXSIZE-1].cur=0;  
} //InitSpace_SL
```

✓ 全部结点都是空闲结点，其中space[0]是空闲链表的头结点（不用于存储数据），其cur域表示第一个可用结点头下标。

算法2.15 , 返回备用空间第一个可用下标

```
int Malloc_SL(SLinkList &space) {  
    i=space[0].cur; //第一个可用结点下标  
    if (space[0].cur) space[0].cur=space[i].cur; //下个可用下标  
    return i;  
} //Malloc_SL
```

算法2.16 , 回收索引为k的结点 , 并设为可用的第一个结点

```
void Free_SL(SLinkList &space) {  
    space[k].cur=space[0].cur; space[0].cur=k;  
} //Free_SL
```

算法2.17 , 求集合运算 $(A-B) \cup (B-A)$

```
void difference(SLinkList &space, int S) {
```

```
    InitSpace_SL(space);
```

```
    S=Malloc_SL(space);
```

```
    r=S;   scanf(m, n);
```

```
    for(j=1;j<=m;++j) { //建链表A
```

```
        i=Malloc_SL(space);
```

```
        scanf(space[i].data);
```

```
        space[r].cur=i; r=i;
```

```
    } //for
```

```
    space[r].cur=0; //r是链表A尾结点
```

S
1

0		8
1		2
2	c	3
3	b	4
4	e	5
5	g	6
6	f	7
7	d	0
8		9
9		10
10		11
11		0

接上页

```
for(j=1;j<=n;++j) {  
    scanf(b); p=S; k=space[S].cur;  
    while(k!=space[r].cur && space[k].data!=b) {  
        p=k; k=space[k].cur; //元素查找, p为k的前驱  
    } //while  
    if (k==space[r].cur) { //元素不存在, 插入新元素  
        i=Malloc_SL(space);  
        space[i].data=b;  
        space[i].cur=space[r].cur;  
        space[r].cur=i;  
    } //if
```

接上页

```
else { //元素已存在，需删除
    space[p].cur=space[k].cur;
    Free_SL(space, k);
    if (r==k) r=p;
} //else
} //for
} //difference
```

S
1

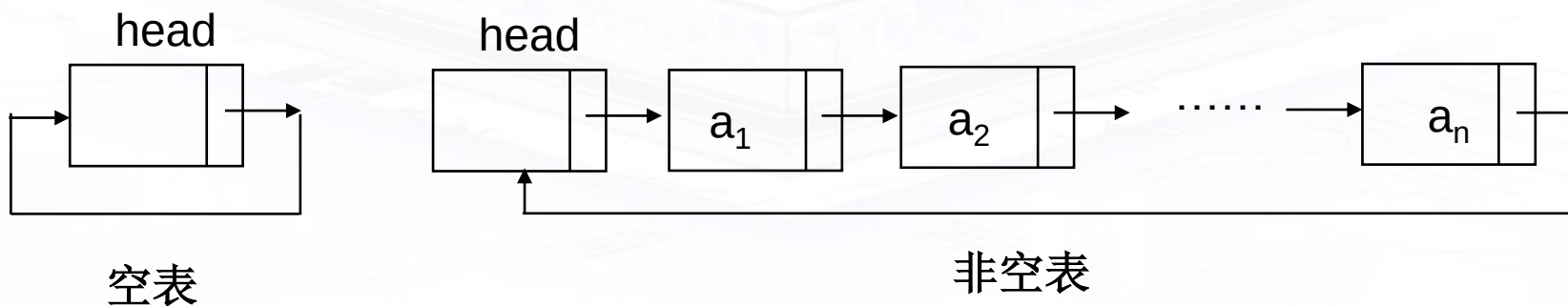
0		6
1		2
2	c	4
3	n	8
4	e	5
5	g	7
6	f	9
7	d	3
8	a	0
9		10
10		11
11		0

2.3.2 循环链表

循环链表(Circular Linked List)：是一种头尾相接的链表。其特点是最后一个结点的指针域指向链表的头结点，整个链表的指针域链接成一个环。

从循环链表的任意一个结点出发都可以找到链表中的其它结点，使得表处理更加方便灵活。

下图是带头结点的单循环链表的示意图。



单循环链表示意图



循环链表的操作

对于单循环链表，除链表的合并外，其它的操作和单线性链表基本上一致，仅仅需要在单线性链表操作算法基础上作以下简单修改：

- (1) 判断是否为空链表：**`head->next==head`** ；
- (2) 判断是否为表尾结点：**`p->next==head`** ；

2.3.3 双向链表

双向链表(Double Linked List) :指的是构成链表的每个结点中设立两个指针域：一个指向其直接前趋的指针域prior，一个指向其直接后继的指针域next。这样形成的链表中有两个方向不同的链，故称为**双向链表**。

和单链表类似，双向链表一般增加头指针也能使双链表上的某些运算变得方便。

将头结点和尾结点链接起来也能构成循环链表，并称之为双向循环链表。

双向链表是为了克服单链表的单向性的缺陷而引入的。



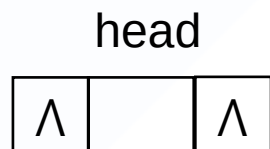
1 双向链表的结点及其类型定义

双向链表的结点的类型定义如下。其结点形式如下所示，带头结点的双向链表的形式如图所示。

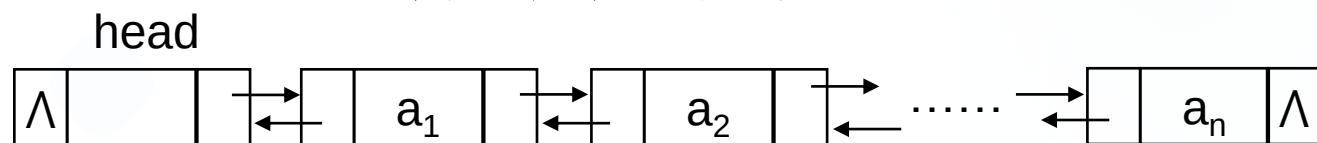
```
typedef struct DuLNode
{
    ElemType data;
    struct DuLNode *prior, *next;
} DuLNode, *DuLinkList;
```



双向链表结点形式



空双向链表



非空双向链表

带头结点的双向链表形式

双向链表结构具有对称性，设p指向双向链表中的某一结点，则其对称性可用下式描述：

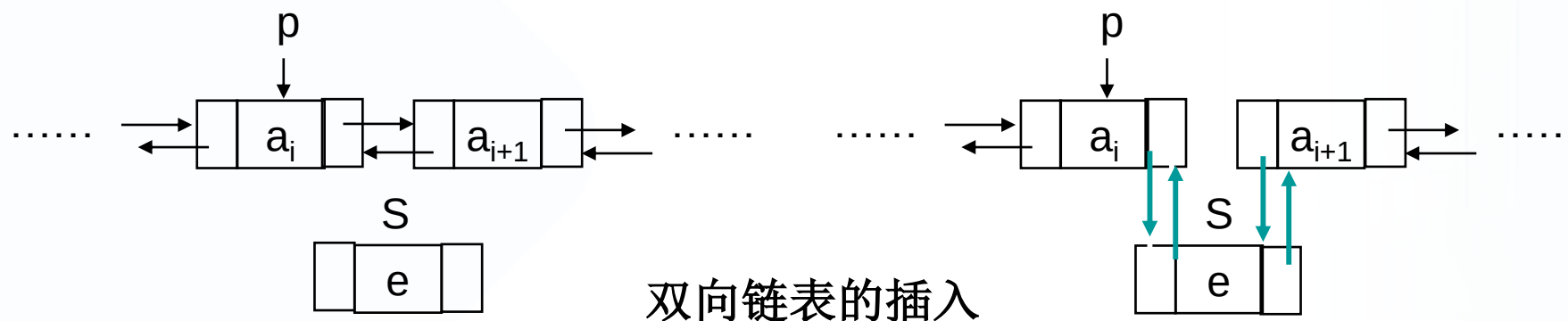
$(p \rightarrow \text{prior}) \rightarrow \text{next} = p = (p \rightarrow \text{next}) \rightarrow \text{prior} ;$

结点p的存储位置存放在其直接前趋结点p->prior的直接后继指针域中，同时也存放在其直接后继结点p->next的直接前趋指针域中。

2 双向链表的基本操作

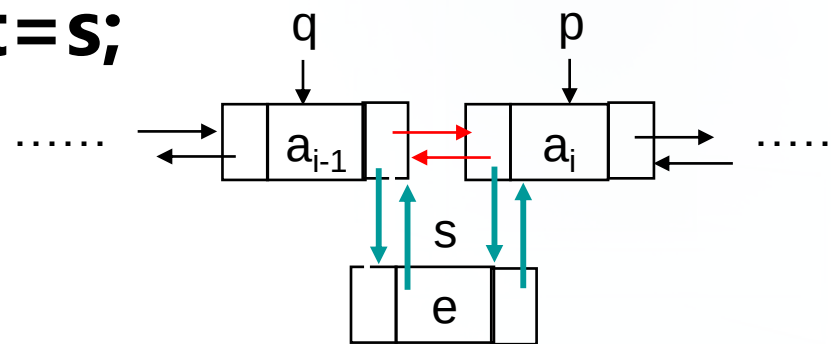
(1) 双向链表的插入

将值为 e 的结点插入双向链表中。插入前后链表的变化如图所示。



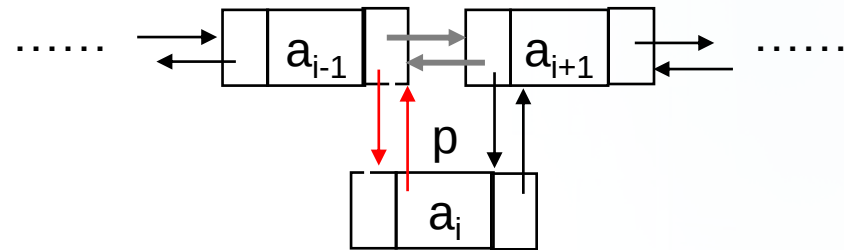
算法2.18 , 双向链表插入

```
Status ListInsert_DuL(DuLinkList &L, int i, ElemType e) {  
    if (!(p=GetElemP_DuL(L, i)) return ERROR; //定位位置i  
    if (!(s=(DuLinkList)malloc(sizeof(DuLNode))))  
        return ERROR;  
    s->data=e;  
    //修改指针的原则：先改新结点，后改原链表；若引入q会更简单。  
    s->prior=p->prior; p->prior->next=s;  
    s->next=p;  
    p->prior=s;  
    return OK;  
} //ListInsert_DuL
```



算法2.19 , 双向链表删除

```
Status ListDelete_DuL(DuLinkList &L, int i, ElemType e) {  
    if (!(p=GetElemP_DuL(L, i)) return ERROR; //定位位置i  
    e=p->data;  
    p->prior->next=p->next; //删除结点的语句顺序随意  
    p->next->prior=p->prior;  
    free(p);  
    return OK;  
} //ListDelete_DuL
```



2.4 一元多项式的表示及相加

1 一元多项式的表示

一元多项式 $P_n(x) = p_0 + p_1x + p_2x^2 + \dots + p_nx^n$, 由 $n+1$ 个系数唯一确定。

在计算机中, 可用线性表 $(p_0, p_1, p_2, \dots, p_n)$ 表示。既然是线性表, 就可以用顺序表和链表来实现。

两种实现方式各有优缺点, 实际应用中以链式居多。



2 一元多项式的相加

不失一般性，设有两个一元多项式：

$$P(x) = p_0 + p_1x + p_2x^2 + \dots + p_nx^n ,$$

$$Q(x) = q_0 + q_1x + q_2x^2 + \dots + q_mx^m \quad (m < n)$$

$$R(x) = P(x) + Q(x)$$

$R(x)$ 由线性表 $R((p_0+q_0), (p_1+q_1), (p_2+q_2), \dots, (p_m+q_m), \dots, p_n)$ 唯一表示。



一元多项式相加的实质是：

- 指数不同：是链表的合并。
- 指数相同：系数相加。和为0，去掉结点；和不为0，修改结点的系数域。

• 以下用**有序**链式存储为例讲解。多项式的项定义如下：

```
typedef struct {  
    float coef;  
    int expn;  
} term, ElemType;  
typedef LinkList polynomial;
```

算法2.22

```
void CreatePolyn(polynomial &P, int n) {  
    InitList(P); h=GetHead(P);  
    e.coef=0.0; e.expn=-1; SetCurElem(h,e);  
    for(i=1;i<=n;++i) {  
        scanf(e.coef, e.expn);  
        if (!LocateElem(P, e, q, (*cmp)())) {  
            if (MakeNode(s, e)) InsFirst(q, s);  
        }  
    }  
} //CreatePolyn
```

算法2.23

```
void AddPolyn(polynomial &Pa, polynomial &Pb) {  
    ha=GetHead(Pa); hb=GetHead(Pb);  
    qa=NextPos(Pa, ha); qb=NextPos(Pb, hb);  
    while (qa && qb){  
        a=GetCurElem(qa); b=GetCurElem(qb);  
        switch (*cmp(a,b)) {  
            case -1: //Pa的当前结点指数小  
                ha=qa; qa=NextPos(Pa, qa); break;  
            case 0: //指数相等  
                sum=a.coef+b.coef;
```

接上页

```
if (sum!=0.0) {  
    SetCurElem(qa, sum); ha=qa; }  
else {  
    DelFirst(ha, qa); FreeNode(qa); }  
DelFirst(hb, qb);  
FreeNode(qb);  
qb=NextPos(Pb, hb);  
qa=NextPos(Pa, ha);  
break;
```


接上页

case 1: **//Pb的当前结点指数小**

DelFirst(hb, qb); InsFirst(ha, qb);

qb=NextPos(Pb, hb);

ha=NextPos(Pa, ha);

break;

} //switch

} //while

if (!ListEmpty(Pb)) Append(Pa, qb);

FreeNode(hb);

} //AddPolyn

结束

谢谢

